hashlock.

Security Audit

Rakurai (DeFi)

December 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Formal Verification Framework	26
Centralisation	42
Conclusion	43
Our Methodology	44
Disclaimers	46
About Hashlock	47

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

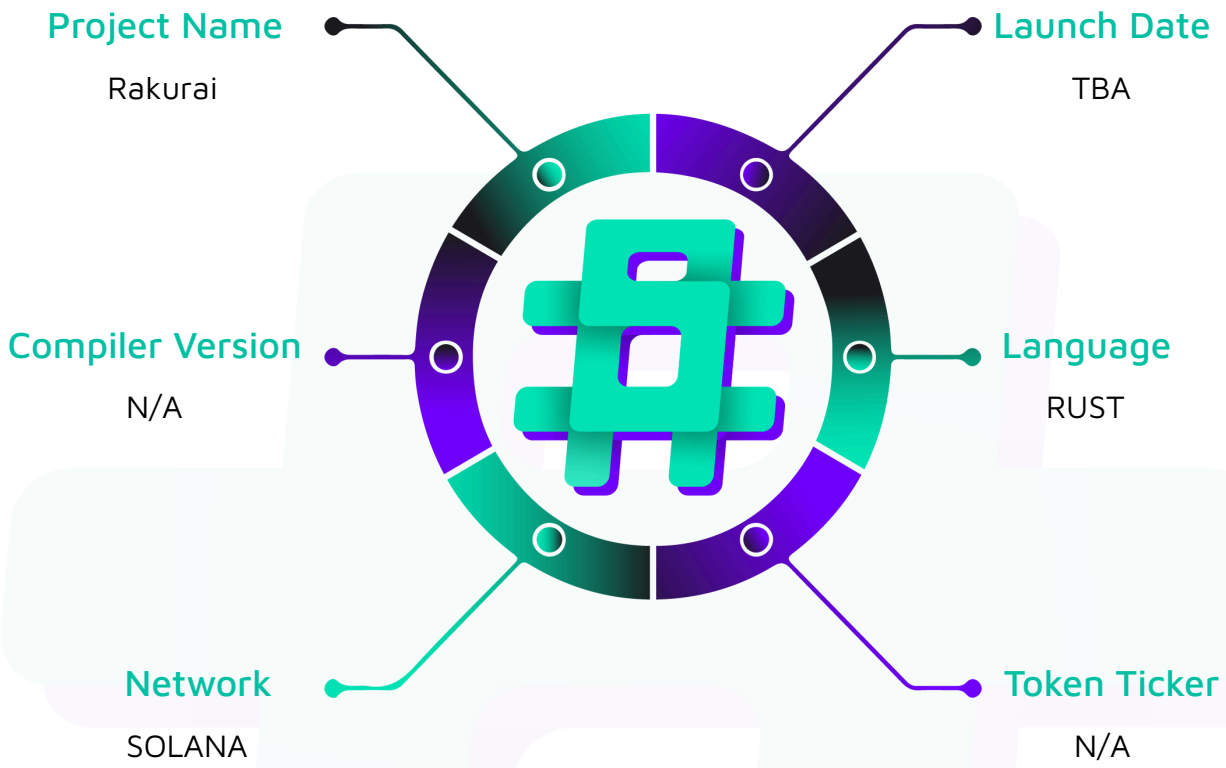
The Rakurai team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

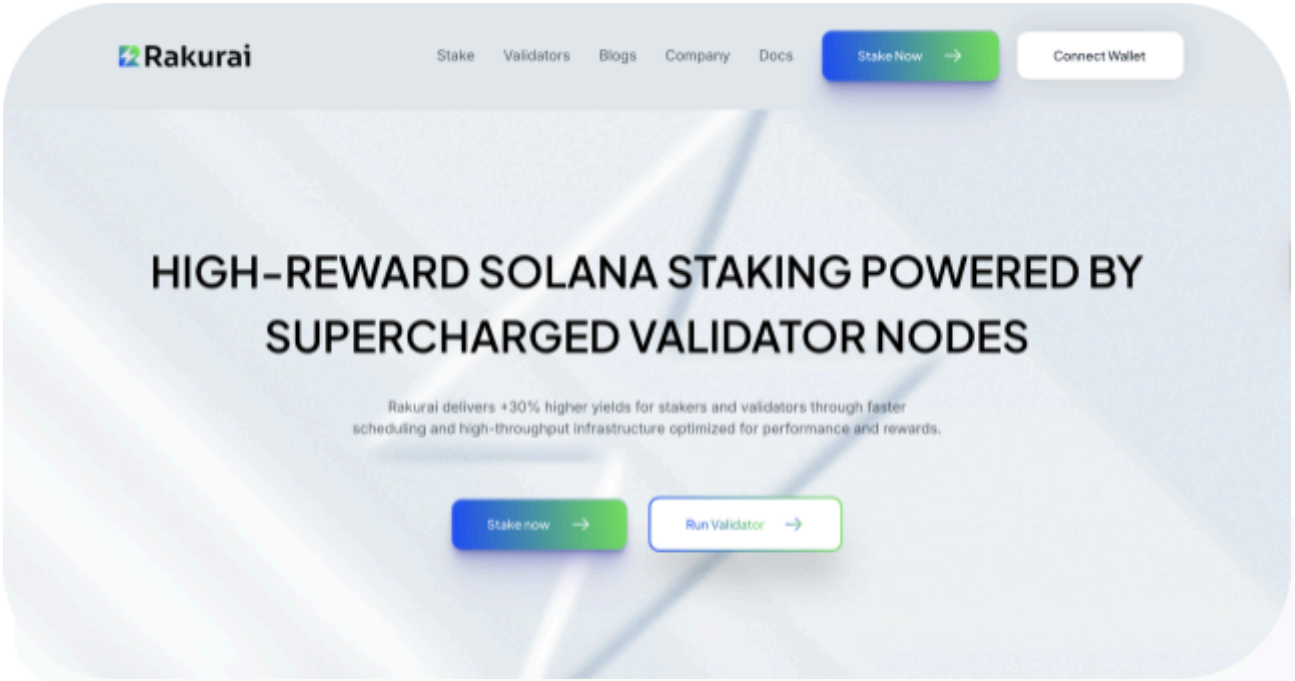
Rakurai is a Web3 infrastructure project (backed by Anagram, Colosseum, P2P, Slow Ventures et al), building a high-performance validator client for Solana. Their focus is on increasing throughput and improving block packing so transactions land more reliably.
One of their biggest strengths is a proprietary scheduler designed to maximize validator rewards without sacrificing TPS and maintaining full compatibility with Solana Foundation Delegation Program (SFDP).
Right now, validators running their client are earning +35% more rewards than the cluster average. Rakurai also enables validators to share this performance boost with their delegators through automated block-reward sharing, resulting in higher yields. All reward distribution is fully automated, with no operational overhead.
Rakurai also offers an LST ($raiSOL) that allows users to stake SOL and earn some of the highest APYs available, with liquidity accessible across Solana DeFi platforms like Sanctum and Jupiter.

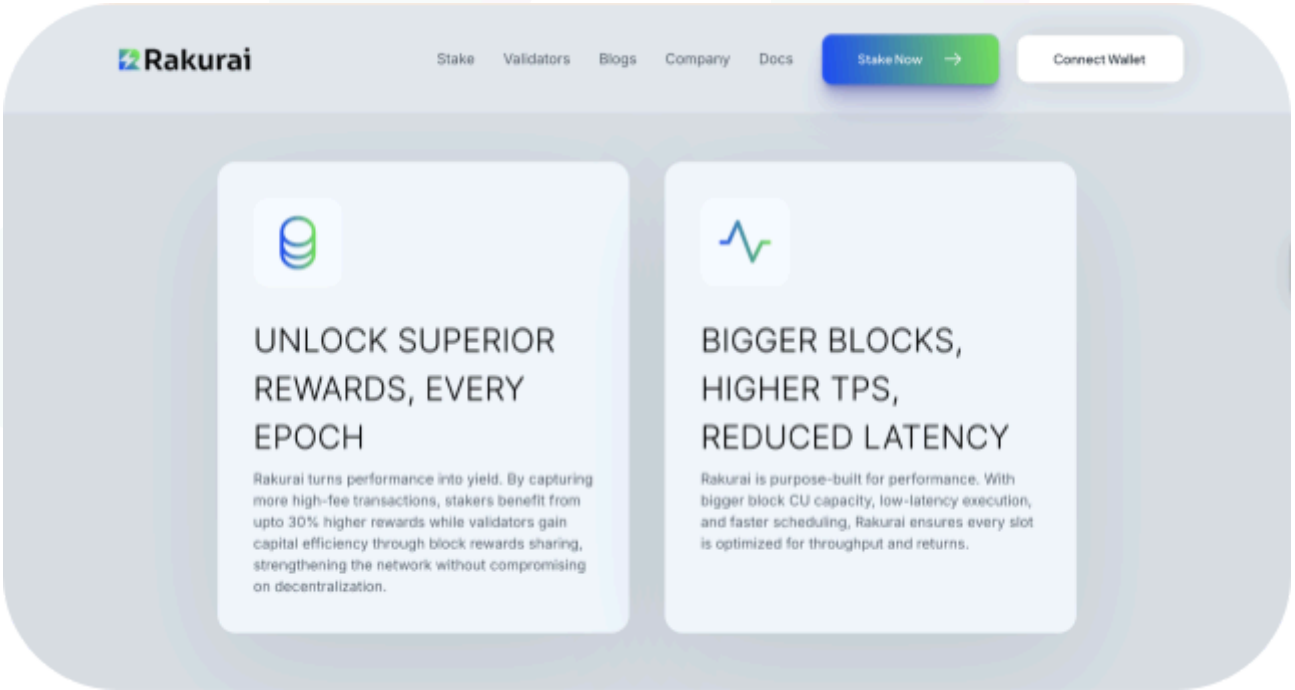
Project Name: Rakurai
Project Type: DeFi
Website: https://rakurai.io/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Rakurai project; the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Rakurai Smart Contracts**
Platform	**Solana / Rust**
Audit Date	**December, 2025**
GitHub Repository	https://github.com/rakurai-io/scheduler_1_0/tree/hashlock-audit-v3.1.10
Audited GitHub Commit Hash	304a06e2d8163b74c0959c92da781a6cb3b6b8d3
Fix Review GitHub Commit Hash	60170b718cde1a5635fae04174966d554c2d20b8

Security Rating

After Hashlock's Audit, we found the system to be **"Secure"**. The solutions all follow complex logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

8 Medium severity vulnerabilities

6 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the smart contracts of the Rakurai project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Rakurai project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] RakuraiTransactionViewReceiveAndBuffer - Precision loss when calculating for accCollateralRewardsPerShare causes depositors to miss out on rewards

Impact

During leader slots (and other non-forward states), validators using the TransactionView receive path can discard most or all inbound transactions, causing severe throughput and liveness degradation in affected slots.

Recommendation

Make `should_parse` true for the states where parsing or buffering is required.

Status

Resolved

Medium

[M-01] SwitchingController - Hardcoded activation verification key is a single point of failure

Impact

A compromise of the signing key used for activation signatures can enable unauthorized activation-state changes to be accepted by every validator running this code, depending on how the on-chain activation account is updated.

Recommendation

Provide a supported key-rotation mechanism and move the trusted key material to a secure, access-controlled configuration source rather than embedding a single immutable key in the binary.

Status

Resolved

[M-02] SchedulerReceiver - Extremely large receiver-side buffer capacity enables memory exhaustion under load

Impact

A transaction flood can push the validator toward very large in-memory queues, triggering Out of Memory errors or swap thrashing.

Recommendation

Reduce the default capacity to a memory budgeted value aligned with end-to-end pipeline limits and add backpressure behavior based on occupancy and memory pressure.

Status

Resolved

[M-03] SchedulerReceiver - Cleanup drains only the main container, leaving `bot_container` resident

Impact

Bot traffic can accumulate across cleanups and cause steadily increasing memory usage, eventually degrading performance or crashing the process.

Recommendation

Extend cleanup to also drain `bot_container`, and ensure both container removals are reflected consistently in drop metrics.

Status

Resolved

[M-04] SchedulerReceiver - Bot-only workload can stall dispatch and strand buffered transactions

Impact

An attacker generating only bot-classified transactions can force a stall where resources are consumed buffering traffic while little or no scheduling progress is made.

Recommendation

Ensure dispatch considers `bot_container` even when the main container is empty, and implement a bounded "drain policy" for bot work rather than returning "no work" when only bot transactions are available.

Status

Resolved

[M-05] Controller - Controller ignores channel disconnect and cannot report liveness correctly

Impact

Upstream crashes or restarts can leave the controller in a state where it appears active but processes no new transactions.

Recommendation

Explicitly handle `TryRecvError::Disconnected` and propagate a shutdown and error signal so disconnections are surfaced and the system can recover properly.

Status

Resolved

[M-06] RakuraiSanitizedTransactionReceiveAndBuffer - Packet batch buffering performs deep clones of batch contents

Impact

Transaction floods can trigger disproportionate memory growth and CPU overhead from repeated deep clones, reducing capacity for legitimate traffic.

Recommendation

Store the original `Arc<Vec<PacketBatch>>`, cloning the `Arc`, not the inner `Vec`, in the buffer, and copy only minimal metadata when necessary.

Status

Resolved

[M-07] SwitchingController - The `send_chunk` clones the workload and drops send errors, enabling silent loss

Impact

Channel failures can cause silent transaction loss and misleading operational metrics.

Recommendation

Move the buffer contents instead of cloning, and treat send failures as actionable errors rather than clearing unsent work.

Status

Resolved

[M-08] ThreadSet - The ThreadSet::as_flag shift causes undefined behavior

Impact

Undefined behavior if invalid `thread_id` values reach `ThreadSet` operations, potentially causing memory corruption or incorrect lock state.

Recommendation

Add `debug_assert!(thread_id < 64)` in `as_flag`, or implement runtime bounds checking that returns a safe default or panics with a clear message.

Status

Resolved

Low

[L-01] Contracts - Panics on missing transaction state enable crash-on-inconsistency DoS

Recommendation

Replace panics with defensive cleanup and log warnings with enough context to debug the root cause.

Response

Rakurai team informed Hashlock that by design, this inconsistency has been eliminated, and this is exactly how it is treated in the standard Agave scheduler as well, which is running the bulk of the Solana network. Since this would be a functionality break if it ever occurs(however remote), we'll move to debug_assert with proper logs in the near future, so that this is present only while testing and in production, and add logic to switch schedulers in failure conditions.

Status

Acknowledged

[L-02] RakuraiReceiveAndBuffer - Drop-reporting path deserializes every dropped packet

Recommendation

Avoid full deserialization for drop reporting where possible, and rate-limit status generation under load.

Response

Rakurai team informed Hashlock that this is important for their Formal verification transaction integrity test, which is essential since the scheduler is closed source.

Status

Acknowledged

[L-03] RakuraiTransactionViewReceiveAndBuffer - Blacklisted-account drops are misclassified as lock-validation errors

Recommendation

Return `PacketHandlingError::BlacklistedAccount` for blacklist matches so the caller records the correct drop reason and metrics.

Status

Resolved

[L-04] Controller - Capacity drops are incorrectly counted as "sanitization" drops

Recommendation

Increment `fetching_summary.num_dropped_on_capacity`, not `num_dropped_on_sanitization`, for capacity-based drops.

Status

Resolved

[L-05] Controller - Controller inserts cloned transactions, increasing memory pressure

Recommendation

Move or share transaction ownership through the pipeline to, for example, `Arc`-backed payloads, rather than cloning transaction objects at insert time.

Response

Rakurai team informed Hashock that the TransactionView enablement has now already optimized the copy since it has a larger chunk of Transaction under an Arc, which never gets copied. Only a little deserialized portion will get copied.

Status

Acknowledged

[L-06] Controller - Relaxed atomic ordering causes stale reads on ARM

Recommendation

Use `Ordering::Acquire` for loads and `Ordering::Release` for stores of `priority_threshold`, or `Ordering::SeqCst` for both if stronger guarantees are needed.

Response

Rakurai team informed Hashlock that the release is used for performance reasons, as the accuracy of the exact priority is less important

Status

Acknowledged

Formal Verification Framework

A Formal Verification (FV) framework was requested to attest the proprietary (in this context specifically non-OSS) Rakurai Solana Scheduler (referred to as the Scheduler) does not:

1. Access and/or leak private (signing) keys,
2. Expose the mempool before transactions are finalized,
3. Perform any internal MEV,
4. Generate reward payout transactions in the non-OSS codebase.

Additionally, activation is performed using a 2-of-2 multisig handshake smart contract, allowing either party to withdraw from the agreement.

Other than static code analysis, performed in previous sections, attestation of the packaged binary is required. Naturally, this suggests that the trust anchor should be set based on the audit report and attestation. Nevertheless, wherever possible, user-verifiability is advised and should be documented by Rakurai in the Scheduler release notes and/or user manual.
As such, FV can only cover a part of the stringent attestation; the rest must be documented for the user for their independent verification. Only when the two are combined is the attestation valid.
The Scheduler is used by a Solana Client (referred to as the Client) by linking it at build time. The Scheduler is built for x86_64 architectures only.

There is a separate section which covers packaging and operating the software

Verifiable claims:
The following serve as the bare minimum for the Scheduler FV framework and are built on the claims presented by Rakurai. As the project evolves, claims may need to be added or removed, depending on the nature of codebase changes:

1. No filesystem access
2. No inbound or outbound network access (inter-thread communication using MPSC)
3. No syscalls beyond a whitelist [RFC]

4. No dynamically loaded code (native to Rust, bar e.g. libssl, libc, et al.)
5. No self-modifying code
6. No access to environment variables beyond a whitelist (e.g. SCHEDULER_CHUNK_SIZE)
7. Deterministic behavior (for synthetic isolated test execution)
8. Reproducible builds
9. Binary library checksum signed (by Rakurai and the auditor), included in the package, release notes, and versioned documentation
10. Attested SBOM (GH attestations present)
11. Sanitized tracing and logging
12. Logging of input and output transaction hashes (count not sufficient)
13. Immutable versioning of the binary library consistent with repository branches/tags

Interpretation and reproducibility:

The claims described above differ in nature. Some can be independently verified by an end-user holding the binary, some pertain to logging/tracing/observability, and some pertain to storage system configuration, which is used for distribution. During detailed discussion, the claims are categorized as either:

- User-verifiable - holders of the binary can use tools to verify claims themself. Additionally, verification steps should be included in the CI part of the pipeline for transparency. The objective of these claims is for the user to independently observe that there are indeed no claimed mechanisms used. This should be used in tandem with SBOM, as these behaviors may be delegated to external dependencies
- Output - logging/tracing/data output produced during runtime. The objective here is twofold:
    - For the user to observe that there are no data transformations going against the claims, and
    - For the user to observe that no sensitive information is leaked.
- Infra config - configuration matters, which can be highlighted to the user. The bottom line here is for Rakurai to prove that measures have been taken in the infrastructure setup, for the claims of this category to hold.

Naturally, all the claims must be satisfied. Failure to satisfy them may lead to reputational damage, even if the codebase performs no malicious actions.

1. **No file system access**

Category: User-verifiable

This is to prove that the library performs no FS I/O, and as such does not access anything sensitive.

Steps to verify:

The list is non-comprehensive, and may be extended. Note that using named pipes is considered as FS I/O; named pipes, however, are an Inter-Process Communication mechanism which mustn't be used for the claims to hold true.

```
nm -D -C <binary> | grep -E "open|read|write|stat|fstat..."

readelf -Ws -C <binary> | grep "fs::"
```

Response:

This is incorporated, you can refer to Appendix here for details.

2. **No network access**

Category: User-verifiable

This is to prove that the library performs no network I/O. As a scheduler, it has inherently no need to do so.

Steps to verify:

The list is non-comprehensive, and may be extended:

```
nm -D -C <binary> | grep -E "open|read|write|stat|fstat..."

readelf -Ws -C <binary> | grep "socket::|std::net|libc::net|..."
```

Response:

This is incorporated, you can refer to Appendix here for details.

3. **No blacklisted syscalls**

Category: User-verifiable

This is an extension of the two previous claims. A list of syscalls that mustn't be used should be included in release notes and/or in user documentation.

Steps to verify:

Look for all the blacklisted calls in the binary:

```
nm -D -C <binary> | grep -E "<blacklisted_syscall_1>|<blacklisted_syscall_2>|..."
```

Response:

This is incorporated, you can refer to Appendix here for details.

4. **No dynamically loaded code**

Category: User-verifiable

This is a special case of (3). When code loads dependencies dynamically, their behavior can be changed, without rebuilding the original code itself. This could potentially lead to undesired and/or behavior.

Steps to verify:

Look for calls to dynamic library loading code:

```
nm -D -C <binary> | grep -E "dlopen|..."
```

Response:

This is incorporated, you can refer to Appendix here for details.

5. **No self-modifying code**

Category: User-verifiable

This is a special case of (3). The code may load a different binary, replacing the application running in the spawned process. This means that even though a binary holds no undesired syscalls, a different binary can be loaded into the running process, completely changing the behavior of the application.

Steps to verify:

Look for calls to exec syscall, which loads binaries into a running process or any logic operating on processes:

```
nm -D -C <binary> | grep -E "exec|..."

readelf -Ws -C <binary>| grep "process::"
```

Response:

This is incorporated, you can refer to Appendix here for details.

6. **No environment variable access (other than whitelisted)**

Category: User-verifiable

This is to verify that the binary does not access any undesired environment variables, which may potentially hold sensitive information. Generally, it is considered a good practice to inject sensitive information using environment variables, as they leave no trace in persistent storage. Obviously, in an ideal scenario, the data held in a variable should be encrypted, and evicted from the memory as soon as it is consumed and no longer needed. That said, if a sensitive environment variable lingers in the shell process that executed the binary, it can be freely accessed by malicious code.

The goal here is to prove that there exists a parity between getenv calls and variable names. The caveat is that the Rust compiler treats constants differently from C/C++ compilers, which place them in the .bss segment of the binary if uninitialized or in .rodata if initialized.

Addressing the issue:

Instead of global consts, use global statics:

```
use std::env::VarError;


static ENV_VAR: &'static str = "ENV_VAR";


fn main() {
    let env_var_value = std::env::var(ENV_VAR).unwrap_or_else(|err| match err {
        VarError::NotPresent => {
            tracing::warn!("{ENV_VAR} not set, defaulting to empty string");
            "".into()
        }
        VarError::NotUnicode(_) => {
            tracing::error!("{ENV_VAR} contains invalid unicode");
            panic!("Non-unicode value in {ENV_VAR}!");
        }
    });
    println!("Value of {}: {}", ENV_VAR, env_var_value);
}
```

Statics are immutable by default, which serves a similar purpose to consts, and, most importantly, since they have a static lifetime, they are of deterministic size, and hence a reference is placed in the .data segment, referring to a literal in .rodata segment. They can only be modified in unsafe code.

Steps to verify:

Look for calls to exec syscall, which loads binaries into a running process or any logic operating on processes:

```
nm -D -C <binary> | grep "ENV_VAR"

nm -D -C <binary> | grep "getenv"
```

Response:

This is incorporated, you can refer to Appendix here for details.

7. **Deterministic behavior**

Category: Output

For a FV method to hold, the output has to be deterministic. Meaning for any input in the domain, there exists exactly one output that $f(D_I) = O_I$, every time a simulation is run.

The best way to prove deterministic behavior if to provide synthetic data and feed it to the system in isolation many times over and then observe that the output is the same every time. Practically, hashes of input and output data are sufficient.

A publicly accessible script and a testing harness should be provided for the library, so that the simulations can be performed by end users, to demonstrate that they produce the same output every time.

Best practices:

- Ship the test harness and simulation script with the application.
- Put it in an open-source repository.
- Add it into the CI/CD pipeline, e.g., nightly builds for simple verification.

Response:

This is incorporated, you can refer to Appendix here for details

.

8. **Reproducible builds**

Category: Output

For an FV method to hold, the same codebase has to produce the same exact output, i.e., the same binary. Cargo produces deterministic builds by default. The toolchain should be explicitly specified, though, to make sure that builds are made with a concrete version of the toolchain. Separately, Cargo.toml manifest has to be as explicit as possible and versioned along with Cargo.lock. This goes in tandem with an attested SBOM, produced by e.g. GH Actions attestations.

Best practices:

Specify toolchain explicitly using rust-toolchain.toml file:

```
cat rust-toolchain.toml

[toolchain]

channel = "1.90"

components = [

"clippy",

"rustfmt",

"rust-src",

"llvm-tools",

"llvm-tools-preview",

]

profile = "minimal"

targets = [

"x86_64-unknown-linux-gnu",

"x86_64-unknown-linux-musl",

]
```

9. Checksum of the binary artifact

Category: Output

A signed checksum of the binary should be produced along with the binary. It is strongly suggested that a signing key for signing the builds is managed by the organization independently of GH Actions.

GH Actions have a limited retention period, and as such, artifacts used for checksum validation may be evicted. Shipping artifact checksum and its detached signature (in .asc format) is considered to be an OSS best practice.

Best practices:

- Host a public key issued and owned by the company on the website, GitHub, etc.

- Compute the checksum in the CI/CD pipeline and put it together with the binary

```
sha512sum target/release/evm-tx-signing-api | awk '{ print $1 }' > scheduler.sha512
```

- Sign the checksum file and store the detached signature together with the binary and checksum

```
gpg --detach-sign --armor scheduler.sha512
```

Response:

This is incorporated, you can refer to Appendix here for details.

10. SBOM attestation

Category: Output

Other than thorough verification of the binary artifact itself, attestation of the Software Bill of Materials is essential for proving that:

- The software did not perform the blacklisted syscalls through third-party dependencies, and
- Vulnerable/compromised dependency versions were updated/removed.

Best practices:

- Use GH Actions attestations along with tools like e.g. GH Dependabot, Snyk, et al.
- Retain versioned Cargo.toml and Cargo.lock manifest files.

Response:

This is incorporated, you can refer to Appendix here for details.

11. Tracing and logging sanitization

Category: Output

Tracing and logging are particularly important in providing verifiable information, which later can be independently verified on-chain. At the same time, utmost care must be taken not to compromise any sensitive information in the logs. As much as logs cannot be used to prove sensitive data is not operated on, they can be used to prove that no censorship or MEV are taking place. This is similar to using an example to mathematically disprove a claim. Naturally, the converse does not hold.

Best practices:

- Adhere to a consistent logging standard.
- Ensure no sensitive information is logged.
- Provide a complete log of one full epoch for user reference, with any and all explanations where necessary.

Response:

Logging is consistent with agave-solana releases, with no sensitive information being logged.

12. Input transaction hash logging

Category: Output

This is a special case of (11). Logging Input transaction hashes is particularly relevant here. Solely logging the number of input vs number of output transactions, as much as important, is not sufficient to prove censorship/swapping is not taking place. Since the transactions must not be disclosed prior to reaching the canonical chain, their hashes should be logged, which can then be verified in

Best practices:

- Log hashes of input transactions (in order).
- Log hashes of output transactions (in order) after on-chain finalization.

Response:

This is incorporated, you can refer to Appendix here for details.

13. Versioning and version immutability

Category: Infra config/output

Consistent versioning is vital for proving the legitimacy of the Scheduler software. Semantic

Versioning (semver) is a de facto standard in OSS. Labeling versions with the stage in the release cycle, as well as individual builds, should be used in bisection methods in case of an incident. For versioning to make sense, it must be immutable. Depending on the method of distribution, one can set the container repository tags as immutable or an object storage policy to WORM. This means that once a version is released, it cannot

be overwritten (or removed, for that matter). E.g., in crates.io, a version can be yanked, meaning it can no longer be downloaded, but it cannot be overridden or removed.
Additionally, build attestations, SBOM attestations (10), along with signed checksums (9)
should be either shipped with the binary or put in the documentation.

Best practices:

- Publish base containers in a container registry that supports tag immutability (e.g. Docker Hub, AWS Elastic Container Registry).
- Publish tarballs in object stores (e.g. AWS S3, Cloudflare R2) which support WORM mode (Write Once, Read Many).
- Sign tags on GitHub.
- Sign all commits on GitHub.

Packaging:

The Scheduler is distributed as a binary package, which is a library that effectively precludes process-level sandboxing. The Scheduler, by itself, cannot be packaged in a container image and distributed as a containerized working tool, which normally would be used to enforce certain behavior.
Nevertheless, the library can be provided in a base image container, which then can be used to build a Solana client. This also allows "vendoring" of the scheduler library.
Since the Scheduler needs a handshake contract, public distribution of the binary is permissible, and opening the source of the container image manifest provides a strong verifiable claim, along with library signature and checksum verification (the library should be signed, similarly to e.g. vendored libcrypto).
The Client with the Scheduler should be packaged at the very least in a container image (e.g. Docker).
A strict seccomp policy, precluding the blacklisted syscalls, should be included in the documentation, so that the container orchestration can preclude the blacklisted syscalls
Lastly, the MUSL toolchain should be chosen over GNU, as it favors minimal, static builds, fully compatible with x86_64.

Host system considerations

The seccomp policy only restricts which syscalls can and cannot be used. Other than that it is completely context-free, which means that there is no headroom for exceptions and as such may be too strict. To additionally narrow down what system resources the Client can access, effectively proving that the Scheduler is not performing anything undesirable, the following should be considered:

- Strict seccomp policy, where applicable
- Strict firewall rules using static (asymmetric) NACLs, not dynamic rules
- Landlocking FS access
- Enforcing strict SELinux policy

All these should be thoroughly documented in the official documentation pertaining to running in the production section and set up in either the base container (where applicable) or container orchestration template (e.g., Docker Compose template, Kubernetes manifest, or some IaC template)

Overall distribution suggestion

At the very least, package the Scheduler in a container image to be used as a base image for building the Client.

Alternatively, the Client with the Scheduler can be distributed as Apache 2.0. License, which applies to the Client, permits this.

Rakurai Formal Verification Response Appendix:

1. **No File I/O**
2. **No network I/O**
3. **No blacklisted syscalls**
4. **Dynamically loaded code**

The Rakurai Scheduler library does not perform or require any file system access, network access, or dynamically load code during its execution. To provide transparency and verifiable assurance, a dedicated test is implemented for users to verify the above-mentioned claims.

This test environment applies seccomp filters to disable file system and network-related system calls before the scheduler is instantiated. If the scheduler attempts to perform any restricted operation, the test will return with a non-zero exit code indicating test failure.

This test is made available through our GitHub repo, and users can choose to run it locally on the latest downloaded binary anytime to verify. The test is also executed as part of the CI pipeline every time a new version is released to transparently provide assurance that none of the above-mentioned calls are part of the code.
Related links:
Scheduler Test Setup Code
CI workflow
Seccomp usage example
Seccomp Documentation
Seccomp filters applied

5. **No self-modifying code**

The Rakurai Scheduler does not execute external binaries or modify its own code at runtime. To independently verify this claim, a static analysis script is set up and is available for users, which excludes the presence of any process loads at runtime.

This test is made available through our GitHub repo, and users can choose to run the static analysis script locally anytime to verify. The test is also executed as part of the CI pipeline every time a new version is released to transparently provide assurance that no execution of external binaries or self-modifying code is present.

6. **No environment variable access**

The Rakurai Scheduler does not access or depend on environment variables for its operation. In particular, Solana does not require sensitive information, such as private keys, to be provided via environment variables.

Given the above, we strongly recommend that users not export any sensitive information/keys in environment variables.

7. **Deterministic behavior**
12. **Input transaction hash logging**

The Rakurai Scheduler exhibits fully deterministic behavior as far as the transactions coming in as input and corresponding transactions going out at the output. To ensure this:

- The Tx In/Out feature records the hashes of all transactions entering and leaving the scheduler. This allows clients to independently confirm that no transactions are censored or altered, ensuring complete traceability. Validators can enable this feature using the --tx-io-check flag at startup and verify using the provided post-processing script. For more information, see Tx In/Out Feature.

- A dedicated test is available for clients to run locally and also on demand in GitHub Actions. This test matches input transaction hashes with output transaction hashes and confirms there is no injection or censoring of transactions inside the scheduler. For more information, see its CI workflow and test script.

9. **Checksum of the binary artifact**
10. **SBOM attestation**

The Rakurai Scheduler ensures its binaries are authentic and verifiable. Each binary has GitHub artifact attestation, providing cryptographic proof of its build provenance. Alongside the binary, a .sha512 checksum and its GPG signature allow anyone to confirm that the downloaded artifact exactly matches the official release. Additionally, the CI/CD pipeline generates an attested Software Bill of Materials (SBOM) which verifies the binary's dependencies as well. Find the CI workflow here.

Users can independently verify the following on the downloaded library:

- GitHub Artifact Attestation using GitHub CLI
- SBOM attestation using GitHub CLI
- Binary checksum following the steps mentioned in the README
    - This checksum verification is in addition to and works independently of GitHub artifact attestation.

Centralisation

The Rakurai project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Rakurai project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Furthermore, the claims presented in the formal verification framework hold, meaning that an independent security researcher can verify that the proprietary software does not perform any activities beyond what the official documentation stipulates. It is strongly suggested that during an independent claim analysis the binary with the Scheduler library is compared with the vanilla node binary, to demonstrate which system calls stem from the node code. Upon reviewing the codebase and conducting binary analysis, Hashlock did not observe any behavior outside what the documentation stipulates.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au